

FenixOS / AnonNet — Raport Bezpieczeństwa v001

Zdecentralizowana sieć P2P z łańcuchem FNX (monero-style), Fenix OS Live ISO z amnezją, własnym GUI, komunikatorem E2E i rdzeniem ukrytych kwot. Raport z pełnego audytu wykonanego na tym snapshocie kodu (nie na obietnicach).

WERDYKT: WSZYSTKO ZIELONE — 31/31 selftestów · 20/20 kontroli statycznych

red-team self_attack: 27 ataków odparzonych · 5 notatek · 0 problemów · build_iso: 56/56 · data: 2026-08-06 14:00 · commit: 742fe4e czat E2E (M4) w GUI Kontakty: rdzen messenger
środowisko audytu: python 3.13.14 · uid=1000, bez roota, bez qemu (Linux kontener)

Co znaczy „zielone” uczciwie

Kod SAM siebie atakuje i broni: każdy moduł ma selftest, a full_audit uruchamia wszystkie 31 jako osobne procesy + 20 kontroli statycznych (sekrety, exec-bity, .gitignore, dokumentacja, czystość drzewa git). To NIE dowód na 'brak 0day' — to dowód, że zaimplementowane mechanizmy obronne naprawdę działają (a dziury, które znaleźliśmy po drodze, mają test-strażnik). Sekcja problemów niżej mówi wprost, czego jeszcze nie ma.

Jak cała sieć działa razem (warstwy 0-7)

Warstwa 0 TOŻSAMOŚĆ

core/identity + core/keystore: username + passkey (Argon2id) → Ed25519+X25519 w kontenerze KS1; passkey-panika otwiera wabik (D24); adres FNXS1 = sig||x pub + checksum (jeden codec core dla portfela i messenger).

Warstwa 1 OS (FenixOS Live ISO)

amnezja (tmpfs, live-build), spoof MAC/hostname/machine-id PRZED siecią (network-pre), nftables FAIL-CLOSED (policy drop x3, skuid 1088, fwmark 51820), panicd D27 (Del+PageUp 2 s → shred+poweroff), sealed payload (Shamir 3z5), tmpfiles /run/fenix 0770 na pliki sterujące RAM.

Warstwa 2 TRANSPORT

net/frame AEAD v2 (chaCha20-Poly1305, nonce per SEQ, replay-okno; drut = zero jawnych markerów — test), camo A (rekordy 4096B + cover), dnscamo D (base32 qname↔TXT), Mullvad WG (kill switch PRZED tunelem; konto tylko z env).

Warstwa 3 SIEĆ P2P

net/fenix_node: mesh gossip tx/bloków (flood anti-echo), sync z chunkingiem, PoW CPU (argon-lite/sha256d), fork-choice najdłuższy ważny łańcuch; demon wystawia IPC /run/fenix/node.ipc (0660, NDJSON, biała lista opów, limity).

Warstwa 4 CHAIN

chain/ledger: konta+nonce (chain+mempool), fee D15 (0.001% burn + 0.055% skarbiec), retarget 144 bloków, mempool cap; username on-chain (D28).

Warstwa 5 PRYWATNOŚĆ

stealth (odbiorca niewidoczny), LSAG ring 5-11 (nadawca w tłumie), key-image anty-double-spend, pool z retencją wydanych jako wabiki; Pedersen+Borromean 64-bit; **3b rdzeń CLSAG**: MLSAG [P, D=C−Σout−fee·H] + blob kwotowy AEAD (AAD=C) — kwoty ukryte, bilans dowiedziony (konsensus v2 w drodze).

Warstwa 6 APP/GUI

gui/fenix_gui (dashboard/portfel/pool/kontakty (resolver + czat E2E M4)/sieć/ustawienia, suwak paranoi D29, badge demona), gui/backend_ipc (most NDJSON z backoffem), gui/pool_panel (adres FNXS1, TX_RING z okna, reszta do siebie), gui/panic_button (hold-2 s D27 + czysty ekran Shift+Esc + vpn.env 0600 w RAM).

Warstwa 7 KOMUNIKACJA M4

app/messenger: kontakt FNXS1, fp TOFU jest tożsamością (username = etykieta), E2E X25519→HKDF→ChaCha + podpis ed25519, seq anty-replay; TofuStore 0600+shred; okno czatu w GUI Kontakty (test 16: drut bez plaintextu, TOFU pin, replay drop, podszywka = OSOBNY kontakt); transport mesh T_MSG off-chain = backlog.

Selftesty: 31/31 zielone (czas, rc)

✓ [CORE	] identity.py	0.1s rc=0
✓ [CORE	] keystore.py (KS1/panic/backoff)	0.3s rc=0
✓ [CORE	] crypto/fenix_crypto.py (E2E+Shamir)	0.1s rc=0
✓ [CORE	] crypto/fenix_wrap.py	0.9s rc=0
✓ [CHAIN	] pow.py (PoW argon-lite)	2.1s rc=0
✓ [CHAIN	] battery: stealth/ring/amount (M5b 1-3)	1.6s rc=0
✓ [CHAIN	] ring_sig.py (nadawca)	1.5s rc=0
✓ [CHAIN	] amount_hide.py (kwoty)	3.7s rc=0
✓ [CHAIN	] miner.py	0.1s rc=0
✓ [CHAIN	] ledger.py (konsensus)	0.1s rc=0
✓ [CHAIN	] tx_ring.py (pool prywatny E2E)	2.8s rc=0
✓ [CHAIN	] usernames.py (D28 on-chain)	0.1s rc=0
✓ [NET	] frame.py (AEAD v2/glina)	0.1s rc=0
✓ [NET	] fenix_node.py (mesh+mining+drut)	5.9s rc=0
✓ [TRANSPORT]	camo.py (padding/cover)	2.0s rc=0
✓ [TRANSPORT]	dnscamo.py (D33 loopback)	9.0s rc=0
✓ [TRANSPORT]	mullvad.py (kill switch)	0.1s rc=0
✓ [OS	] spoof.sh --selftest	1.2s rc=0
✓ [OS	] fenix_panicd.py (D27)	0.0s rc=0
✓ [OS	] fenix_boot.py (bariera D24)	0.2s rc=0
✓ [OS	] fenix_payload.py (sealed)	0.1s rc=0
✓ [OS	] build_iso.sh --selftest (56)	1.7s rc=0
✓ [GUI	] fenix_gui.py (D28/D29 + panic + vpn.env + IPC + Pool + czat E2E)	0.7s rc=0
✓ [GUI	] panic_button.py (D27 w GUI, hold-2s)	0.1s rc=0
✓ [GUI	] backend_ipc.py (most GUI+demon, E2E)	0.6s rc=0
✓ [GUI	] pool_panel.py (M7c: TX_RING z GUI, E2E)	16.4s rc=0
✓ [APP	] messenger.py (M4: E2E+TOFU+impersonacja)	0.1s rc=0
✓ [CHAIN	] clsag.py (3b rdzeń: ukryte kwoty MLSAG)	10.0s rc=0
✓ [APP	] exchange.py (giełda+PSC D32)	0.1s rc=0
✓ [APP	] xmr_swap.py (D34)	2.0s rc=0
✓ [TOOLS	] self_attack.py (red team)	2.7s rc=0

Kontrole statyczne: 20/20

- ✓ zero sekretów PEM w repo
- ✓ .gitignore: *.key
- ✓ .gitignore: secrets/
- ✓ .gitignore: *.iso
- ✓ .gitignore: __pycache__
- ✓ .gitignore: build/
- ✓ exec bit: os/spoof.sh
- ✓ exec bit: os/build_iso.sh
- ✓ exec bit: os/verify_iso.sh
- ✓ exec bit: os/includes.chroot/usr/local/bin/fenix-vpn
- ✓ exec bit: os/includes.chroot/usr/local/bin/fenix-netup
- ✓ docs: docs/decyzje_architektury.md
- ✓ docs: docs/threat_model.md
- ✓ docs: docs/ToS.md
- ✓ docs: docs/fnx_spec.md
- ✓ docs: docs/legal/FENIX_Dokument_PL.md
- ✓ docs: docs/legal/FENIX_Dokument_EN.md
- ✓ docs: docs/legal/FENIX_Dokument_RU.md
- ✓ GUI zaimplementowane (M7)
- ✓ git: drzewo czyste (poza regenerowanym raportem ataków)

Błędy ZNALEZIONE w tej iteracji i NAPRAWIONE (każdy ma test-strażnik)

Zasada projektu: test najpierw łAPIE błąd, pokazujemy go wprost, naprawiamy, powtórzony przebieg. Poniżej pełna lista z tej rundy — z P0 na początku.

[P0-KRYTYCZNY] net/fenix_node.py

- Python -m net.fenix_node ZAWSZE odpalał selftest — nawet gdy systemd wołało z flagami demona. Fenix-node.service na ISO nigdy nie wystartowałby prawdziwej sieci P2P.
- naprawa: poprawka kodu + strażnik w build_iso selftest (iso już tego pilnuje)

[WYSOKI] gui/pool_panel.py (test)

- Selftest liczył nonce ręcznie i zderzył się z add_tx (chain+mempool) — pokazało, jak ważny jest wzorzec nonce z ledgera, nie z pamięci.
- naprawa: poprawka w teście (produkcja była czysta)

[WYSOKI] app/messenger.py

- E2E szyfrowało DH z kluczem NADAWCY zamiast ODBIORCY — nikt nie byłby w stanie przeczytać wiadomości (bezpiecznie FAIL, nie przeciek).
- naprawa: poprawka kodu; test E2E łapie

[ŚREDNI] app/messenger.py

- LocalTransport.poll_envs padał z KeyError, gdy odebrano przed pierwszą wiadomością (brak kolejki).
- naprawa: poprawka kodu (setdefault)

[WYSOKI] chain/clsag.py

- MLSAG: sig.c0 = c[idx] podpisującego zamiast c[0] (konwencja z chain/ring_sig) — weryfikator zaczynał od złego wyzwania: ŻADEN podpis by nie przeszedł.
- naprawa: poprawka kodu + wyzwanie z indeksem j (jedna szkoła)

[WYSOKI] chain/clsag.py

- Podpis nad kanonem z placeholderów ≠ kanon liczony przez weryfikator (ki brał się do msg po podpisie).
- naprawa: poprawka kodu: ki PRZED policzeniem msg

[NISKI] workspace

- Snapshot środowiska gubi exec-bity skryptów (5 plików) → kontrole exec-bit czerwone + build_iso wymagające -x na fenix-gui padało.
- naprawa: rytuał chmod +x przed audytem/commitem (udokumentowane)

[NISKI] gui/fenix_gui.py (test)

- Test oczekiwał słowa 'IPC' w komunikacie o martwym VPN/miningu — kontrakt to (False, powód z demona).
- naprawa: poprawka w teście (kontrakt ujednolicony)

Wnioski z rundy: najgroźniejszy był P0 dispatch demona (sieć na ISO nigdy by nie wstała) — złapany przy okazji IPC, gdy CLI-demona sprawdzono „faktycznie” zamiast tylko selftestów. Lekcja: każdy plik-demon dostaje test uruchomienia z argv.

Szyfrowanie — stan obrony (warstwa → algorytmy → wynik ataków)

tożsamość+keystore

- Argon2id + KS1 AEAD; wabik panic
- ✓ self_attack: brute/flip/spray — odparzone (27 odparzonych ogółem)

ramki mesh

- X25519→HKDF→ChaCha20-Poly1305; nonce per SEQ; replay-window
- ✓ drut: 588 rekordów 4096B, ZERO markerów (frame/node selftest)

camo/dnscamo

- rekordy stałe jak TLS; DNS base32 qname↔TXT
- ✓ loopback E2E; profile A/D przechodzą

messenger E2E

- X25519 eph → HKDF('FNXM1') → ChaCha20-Poly1305 (AAD); podpis ed25519
- ✓ impersonacja: fp wykrywa; replay drop; flip bitu = bad_aead

pool/ring

- ed25519 LSAG + stealth ed25519 + Pedersen + Borromean 64-bit
- ✓ double-spend odrzut; $\Sigma = \Sigma_{out} + fee$; druk FNX odrzut

clsag 3b

- MLSAG 2-kolumnowy + blob AEAD (AAD=C)
- ✓ fee-klamstwo/ki-podmianka/rp-zepsuty → odrzuty; payload bez amt

RNG platformy

- os.urandom
- ✓ test E: 512 KB — odparzone

anti-forensic

- shred 2× nadpis (keystore/tofu/vpn.env), drop_caches, sysrq
- ✓ panicd/dry+real shred — PASS

Spoof (MAC / IP / inne) — analiza

os/spoof.sh: losowy MAC (02: lokalny+unicast, walidator odrzuca multicast/globalne), hostname fenix-<8hex>, machine-id 32hex — generowane PRZED siecią (network-pre.target), 200/200 unikalne w selftestach. Skutek: komputer z ISO nie da się skorelować po LAN (ta sama kawiarnia wczoraj/dziś to inna maszyna). Publiczny IP: tym nie zajmuje się spoof (to nie jego warstwa) — publiczny IP zasłania Mullvad WG (paranoia C z kill switchem) i/lub camo+mesh; bez VPN Fenix pokazuje IP hosta, co jest UDOKUMENTOWANĄ granicą profilu A/B, nie dziurą. DHCP/hostname UUID: losowane per boot; IPv6 EUI-64 dziedziczy z randomizowanego MAC.

Problemy OTWARTE (uczciwie: co jeszcze nie działa / nie jest zweryfikowane)

P2 [WYSOKA] sieć

- Seeds = placeholder DEV (203.0.113.x). Nowy node bez nich nie dołączy do realnej sieci.
- plan: właściciel: ≥ 3 VPS jako seed-nody; podmiana /etc/phenix/seeds.list

P3 [WYSOKA] ISO/QA

- ISO nie bootowane w qemu/kvm (brak w sandboxie). build_iso selftest = 56/56 statycznie, ale to nie boot.
- plan: live-build + qemu na maszynie właściciela

P4 [ŚREDNIA] prywatność

- Kwoty: rdzeń 3b (CLSAG) GOTOWY i testowany; KONSENSUS payloadu v=2 w ledgerze jeszcze nie wdrożony (na dziś on-chain kwoty 3a jawne).
- plan: następny kamień M5b: retencja C w poolu + rejestr ki + migracja outputów 3a

P5 [ŚREDNIA] ekonomia

- BLOCK_REWARD/halving = wartości robocze (fnx_spec §12); skarbiec = DEV wallet.
- plan: decyzja właściciela: emisja + zimny multisig D14

P6 [ŚREDNIA] blockchain

- Ledger tylko w RAM — brak chain.dat (restart = pełny resync).
- plan: snapshot/disk z weryfikacją

P7 [ŚREDNIA] giełda/swap

- Paysafe/monero-rpc gatewaye = kontrakty (NotImplemented); hosting deska poza kodem.
- plan: D32/D34: umowa + VPS

P8 [ŚREDNIA] transport

- DNS-camo wymaga własnej domeny-mostu (zadanie właściciela). Loopback E2E działa.
- plan: rejestracja domeny + DnsBridgeServer

P9 [ŚREDNIA] ISO

- dm-verity nie wdrożone.
- plan: backlog ISO: verity + podpis GPG offline

P10 [ŚREDNIA] AI

- AI-Sentry (M6) — radar administracji nie istnieje.
- plan: po GUI; reguły warstwa 1

P11 [NISKA] krypto

- Rangeproof Borromean 6.3 KB/out (bloki cięższe).
- plan: bulletproofs backlog D31

P12 [NISKA] transport

- snowflake_fnx/onion-relay (M2) nie istnieją; T_MSG (ephemeral gossip pod messenger) = nowy punkt M2/M4.
- plan: M2/M3 etapy

P13 [NISKA] D28

- Avatar = tylko ref-hex; blob-store nie wdrożony.
- plan: M4+ blob-store

P15 [INFO] Mullvad

- Prawdziwe up() (root+konto+sieć) nie testowane — offline 10/10.
- plan: QA na maszynie właściciela

P17 [ŚREDNIA] GUI/QA

- WIDOK GUI (Tkinter) nie testowany pod X11/openbox — cała logika kontrolerów headless ✓ (fenix_gui 16/16), widok strukturalnie gotowy.
- plan: QA na ISO z openbox

P18 [ŚREDNIA] messenger

- Mesh T_MSG + kolejka offline + ratchet PFS (M4b) — rdzeń E2E + okno czatu w GUI działają na LocalTransport (dev); brakuje przekaźnika w prawdziwej sieci.
- plan: relay T_MSG w fenix_node + drut-test bez markerów

P19 [NISKA] messenger

- TofuStore: plik 0600+shred, ale NIE szyfrowany hasłem (kontakty czytelne dla kogoś z RAM/dysku).
- plan: KS1-wrap na tofu + ranking kontaktów

P20 [NISKA] IPC/QA

- Żywy przebieg demon+GUI (2 procesy) nie testowany — IPC sprawdzony E2E z klasą Node i w testach CLI.
- plan: QA na ISO: systemctl fenix-node + fenix-gui

P21 [ŚREDNIA] messenger

- Mapowanie username→kontakt FNXS1 on-chain nie wdrożone (dziś kontakt wymieniany ręcznie/drugim kanałem).

→ plan: TX_CONTACT_REF + resolver w GUI

Jak to uruchomić u siebie

```
selftest całości:      python3 tools/full_audit.py    (raport JSON → /tmp/fenix_audit.json)
red team:              python3 tools/self_attack.py
demon (2 procesy, test): python3 -m net.fenix_node --port 45992 --ipc /tmp/fenix.ipc
...a w drugim oknie GUI: python3 -m gui.fenix_gui --demo    (na maszynie z X11)
messenger demo:       python3 -m app.messenger --demo
klasa 3b demo:         python3 chain/clsag.py    (rdzeń – konsensus w follows)
budowa ISO (Debian):   bash os/build_iso.sh    → potem qemu/kvm smoke boot
regeneruj TEN raport:  python3 tools/make_rap001.py    (po full_audit)
```

Czego NIE testowano (uczciwe granice tego raportu)

- widok GUI pod X11/openbox (logika kontrolerów 15/15 headless PASS; widok = P17)
- boot ISO w qemu/kvm + live-QA Mullvad z kontem (P3/P15)
- pełny mesh demon↔demon z GUI dwoma procesami (P20) — komponenty E2E tak
- obciążenia >sandbox: kopie 1000 node'ów, fragmentacja sieci, czas godzin (M9 QA)

Hasło raportu: projekt NIE deklaruje 'brak dziur 0day' — deklaruje i udowadnia, że (a) każdy obronny mechanizm jest testowany SAM przeciw sobie, (b) znalezione błędy są naprawiane z testem-strażnikiem, (c) lista problemów otwartych jest jawna, bez makijażu. To jest sposób, w jakim projekt zasługuje na zaufanie.